

3.1 Task 1: Deriving the Private Key

1. To calculate the length of n , I need to use the BIGNUM library to convert p and q to binary, multiply them, and find the length (or # of bits) of n . The BIGNUM library thankfully has functions for most of this, and I could do this in only a few steps:

```
#include <stdio.h>
#include <openssl/bn.h>

int main() {
    BN_CTX *ctx = BN_CTX_new();

    BIGNUM *p = BN_new();
    BIGNUM *q = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *n = BN_new();

    BN_hex2bn(&p, "5adb09bdbee704d048da9");
    BN_hex2bn(&q, "14738741a1260adf78815");
    BN_hex2bn(&e, "10001");

    BN_mul(n, p, q, ctx);

    printf("n length: %d\n", BN_num_bits(n));
}
```

Running the code gave me the bit length of modulus n , which was 163:

```
[02/12/26] seed@VM:~/A2$ vim t1.c
[02/12/26] seed@VM:~/A2$ gcc t1.c -o t1 -lcrypto
[02/12/26] seed@VM:~/A2$ ./t1
n length: 163
```

2. To calculate the private key, I need to use Euler's formula on p and q , and then calculate $(e^{-1}) \pmod{\phi(n)}$. This can be done relatively easily by modifying p and q (subtracting 1 from each), and then using the BIGNUM function `BN_mod_inverse()`:

```
// p2
```

```
BIGNUM *d = BN_new();  
BIGNUM *phi = BN_new();  
BN_sub_word(q, 1);  
BN_sub_word(p, 1);  
BN_mul(phi, p, q, ctx);
```

```
BN_mod_inverse(d, e, phi, ctx);
```

```
printf("private key d: %s\n", BN_bn2hex(d));
```

```
[02/12/26] seed@VM:~/A2$ vim t1.c
```

```
[02/12/26] seed@VM:~/A2$ gcc t1.c -o t1 -lcrypto
```

```
[02/12/26] seed@VM:~/A2$ ./t1
```

```
n length: 163
```

```
private key d: 0124898EC0F31E5F67D72E4017BDDE4723A3DCB7C1
```

3.2 Task 2: Encrypting a Message

First, I converted the ASCII text to hexadecimal using the suggested library, and turned “i<3crypto” into “693c3363727970746f”

```
[02/12/26]seed@VM:~/A2$ python
Python 2.7.18 (default, Dec  9 2024, 19:35:20)
[GCC 9.4.0] on linux2
Type "help", "copyright", "credits" or "license" for more
>>> import binascii
>>> print(binascii.hexlify(b"i<3crypto").decode("ascii"))
693c3363727970746f
>>> exit()
```

From here, I converted the hexadecimal to binary, and ran the BN_mod_exp() function, which calculates $m^e \bmod(n)$, which is the formula for generating a ciphertext. The resulting program gave me a ciphertext of 01370B0B64176F4EC06B0D590B369857203050E479.

```
// e initialized and n calculated above

BIGNUM *m = BN_new();
BIGNUM *result = BN_new();

BN_hex2bn(&m, "693c3363727970746f");
BN_mod_exp(result, m, e, n, ctx);

printf("ciphertext: %s\n", BN_bn2hex(result));

[02/12/26]seed@VM:~/A2$ ./t2
ciphertext: 01370B0B64176F4EC06B0D590B369857203050E479
```

3.3 Task 3: Decrypting a Message

Here, I used the same function `BN_mod_exp()` but used different values to perform a decryption, rather than an encryption. Since the decryption formula is $m = c^d \bmod(n)$, and I have access to all of these values, it was simple to plug them in and run the program.

The resulting plaintext in hexadecimal was “73776F726466697368”, which converted to ASCII gave me “swordfish”.

```
// d and n are same values as previously calculated
BIGNUM *cipher = BN_new();
BN_hex2bn(&cipher, "12e6c403952e461afd626e2c81d7b53d0cfe043f3");

BIGNUM *result = BN_new();

BN_mod_exp(result, cipher, d, n, ctx);

printf("plaintext: %s\n", BN_bn2hex(result));

[02/12/26]seed@VM:~/A2$ ./t3
plaintext: 73776F726466697368
[02/12/26]seed@VM:~/A2$ python
Python 2.7.18 (default, Dec  9 2024, 19:35:20)
[GCC 9.4.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> import binascii
>>> print(binascii.unhexlify(b"73776F726466697368").decode("ascii"))
swordfish
```

3.4 Task 4: Signing a Message

100: 49206f776520796f752024313030

200: 49206f776520796f752024323030

1. Here, I generated the hex values of the original message “I owe you \$100” to generate the hexadecimal value 49206f776520796f752024313030. I also found the hexadecimal of “I owe you \$200” for part 3

```
>>> print(binascii.hexlify(b"I owe you $100").decode("ascii"))
49206f776520796f752024313030
>>> print(binascii.hexlify(b"I owe you $200").decode("ascii"))
49206f776520796f752024323030
```

For signing this plaintext, I generated a signature by using the BIGNUM function `BN_mod_exp()`, with input values to calculate $m^d \bmod n$:

```
// d and n already calculated

BIGNUM *plain = BN_new();
BN_hex2bn(&plain, "49206f776520796f752024313030");

BIGNUM *sig = BN_new();
BN_mod_exp(sig, plain, d, n, ctx);
printf("signature: %s\n", BN_bn2hex(sig));
```

After running this program with both the plaintexts, I received the two corresponding signatures:

```
[02/12/26]seed@VM:~/A2$ ./t4
hex:      49206F776520796F752024313030
signature: 032A9C44DE33D475AF40C0F56B8AA024448F25E69F
[02/12/26]seed@VM:~/A2$ ./t4
hex:      49206F776520796F752024323030
signature: 02FF9B900B701412DE3A6620CB96E7F6216C589157
```

2. For the exponent, I used the private key. This way, there was still security to the signature.
3. By comparing the signatures for both messages (image above), I can see that even though the messages are almost identical, there is no observable pattern in the signatures. The only thing I could maybe consider to be a pattern is that the first two digits of the signature are similar, and the 5th and 6th digits are similar, but this is most likely a coincidence.

3.5 Task 5: Verifying a Signature

Here is the code I used to generate a plaintext from Alice's signature, which generated me the plaintext (in hexadecimal) of "4C61756E636820612D697373696C652E". To get this, I used the BN_mod_exp() function again with the signature, e, and n as inputs, using the formula $m = s^e \bmod n$:

```
int main() {
    BN_CTX *ctx = BN_CTX_new();

    BIGNUM *e = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *sig = BN_new();

    BN_hex2bn(&e, "10001");
    BN_hex2bn(&n, "ae1cd4dc432798d933779fbd46c6e1247f0cf1233595113aa51b450f18116115");
    BN_hex2bn(&sig, "643d6f34902d9c7ec90cb0b2bca36c47fa37165c0005cab026c0542cbdb6802f");

    BIGNUM *plain = BN_new();
    BN_mod_exp(plain, sig, e, n, ctx);

    printf("plain: %s\n", BN_bn2hex(plain));

    [02/12/26]seed@VM:~/A2$ ./t5
plain: 4C61756E63682061206D697373696C652E
```

By converting this text into ASCII, I can see that this indeed would be the signature for the message "Launch a missile."

```
[02/12/26]seed@VM:~/A2$ python
Python 2.7.18 (default, Dec 9 2024, 19:35:20)
[GCC 9.4.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> import binascii
>>> print(binascii.unhexlify(b"4C61756E63682061206D697373696C652E").decode("ascii"))
Launch a missile.
>>> exit()
```

By changing the second last digit in the signature from a 2 to a 3, I can first observe that the plaintext generated is much longer. When trying to convert the plaintext into ASCII, python throws an error, as it receives a hexadecimal value outside of the ASCII range (128).

```
[02/12/26]seed@VM:~/A2$ ./t5
signature: 643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6803F
plain    : 91471927C80DF1E42C154FB4638CE8BC726D3D66C83A4EB6B7BE0203B41AC294
[02/12/26]seed@VM:~/A2$ python
Python 2.7.18 (default, Dec 9 2024, 19:35:20)
[GCC 9.4.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> import binascii
>>> print(binascii.unhexlify(b"91471927C80DF1E42C154FB4638CE8BC726D3D66C83A4EB6B7BE0203B41AC294").decode("ascii"))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
UnicodeDecodeError: 'ascii' codec can't decode byte 0x91 in position 0: ordinal not in range(128)
```

3.6 Task 6: Cracking an RSA Private Key

1. Naivest approach:

First, I attempted to write a python script to look for factors, but after running it for 15 minutes I decided it wasn't worth the time and switched to using a library. This script loops through all odd numbers from $\sqrt{n}-1$ (-1 is necessary since $\sqrt{n}$ was even in this case), and checks to see if they are a factor of n:

```
import math
import time

n = 0x1e6f1558fe63761406be065a9e07b060f8360b2725b09f4071
start = int(math.sqrt(n)-1)
start_time = time.time()

for i in range(start,0,-2):
    if (n%i == 0):
        print("Found one!")
        print(i)
        break

end_time = time.time()
elapsed = end_time - start_time
print(f"Done in {elapsed:.6f} seconds")
```

Next, I decided to not reinvent the wheel and implemented a library to do the heavy lifting. I chose the sympy library, as it seemed simple:

```
import time
from sympy import factorint
n = 0x7c5cfe617c286a27ffc10ecf88a8d35ebbf1e30320af
start_time = time.time()
factors = factorint(n)
print(factors)
end_time = time.time()
elapsed = end_time - start_time
print(f"Done in {elapsed:.6f} seconds")

[02/12/26]seed@VM:~/A2$ python3 crack1_2.py
{158314859986381280032123783: 1, 293906828378037345736457113: 1}
Done in 454.093956 seconds
```

This ended up taking about 7.5 minutes to complete.

Next, I used the rsacracker library for 2, and 3:

2. (Note, 4 of the initial ‘10 seconds’ were used to initialize connections to prime number databases)

```
[02/13/26]seed@VM:~/A2$ rsacracker -n 191036808839073752998059970737347860342917676432172452364401 --factors
Running [00:00:06] [=====>] 32/41 Elapsed time: 10.81103245s

Succeeded with attack: factordb
Factors of n:
p = 335403766841212825761913864577
q = 569572639682113597061986473713
```

```
[02/13/26]seed@VM:~/A2$ rsacracker -n 215653767958005306447782312224930614867177974189983834711311896825161936664
15838029014125893 --factors
Running [00:00:00] [=====>] 11/41
Elapsed time: 529.123986ms
Succeeded with attack: factordb
Factors of n:
p = 4343374697891065474200031842625303410093845713
q = 4965120049686628174701117715736096660754359861
```

- 3.
4. I did end up switching from a slightly inefficient library written in python, to a much much more efficient rust package. I know that Rust runs much faster than Python, so when python was taking a long time to run I looked for packages that were written in lower-level languages, such as C or Rust. From here, the time dropped from 450 seconds down to below 7. In fact, the larger and supposedly more difficult to crack number was cracked much more quickly, at only 0.529 seconds, compared to the 6 seconds of the shorter one. I assume this is because both numbers were listed in the “factordb” database it checks before looking to compute it itself, and it simply found the larger number sooner. My computer uses an Intel i5-10600k CPU (with the VM having 2/12 threads allocated), and 2048MB/16384MB of RAM used. Ordinarily, I would assume that larger numbers would be harder to crack, as they would be harder to discover and factor.
- a. Using an external database sort of cheats the “use a better algorithm” part of this question as it doesn’t show that rust libraries *actually* compute primes factor than python libraries, but unfortunately there isn’t an option to disable external databases from the factorization function, and our assignment wouldn’t have a never-before-seen prime number.